

0.面向对象基础

0.什么是面向对象编程和面向过程编程

面向对象编程（OOP）

面向对象会先抽象出对象，然后用对象执行方法的方式解决问题。

面向对象编程是一种以“对象”为中心的编程方式。在OOP中，数据和**操作数据的方法被封装在一起**，形成一个单元，即对象。对象是类的实例，而类则定义了对对象的属性和行为。OOP强调以下概念：

- **封装**：将数据和与数据相关的操作绑定在一起，隐藏内部状态，仅通过接口暴露有限的操作。
- **继承**：允许创建新的类，这些类从现有类那里继承属性和方法，并可以扩展或重写它们。
- **多态**：同一接口可以在不同的对象上表现出不同的行为，这允许程序更加灵活地处理不同类型的对象。

面向过程编程

面向过程把解决问题的过程拆成一个个方法，通过一个个方法的执行解决问题。

面向过程编程是一种更传统的编程范式，它主要**关注于如何执行一系列的步骤来解决问题**。在面向过程编程中，**程序被组织为一系列函数或过程，每个函数或过程完成特定的任务**。面向过程编程的特点包括：

- **模块化**：将程序分解成独立的、可重用的模块或子程序。
- **数据和行为分离**：数据和对数据的操作通常是分开的，函数通常只处理传入的数据参数。
- **顺序执行**：程序通常按照预定义的顺序执行一系列步骤。

比较

- **OOP** 更适合处理复杂、大型项目，因为它提供了更好的代码复用性、灵活性和维护性。OOP也更容易实现抽象和模拟现实世界中的事物和关系。
- **面向过程编程** 在小型项目或不需要复杂数据结构的情况下可能更简单、直接。它也常用于系统级编程，如操作系统内核开发。

1.哪些代码设计看似是面向对象，实际是面向过程的？

即便在面向对象编程中，我们也并不是完全排斥面向过程风格的代码。只要它能为我们写出好的代码贡献力量，我们就可以适度地去使用。

1.滥用getter、setter方法

有同事定义完类的属性之后，就顺手把这些属性的getter、setter方法都定义上。这样的做法我是非常不推荐的。它违反了面向对象编程的封装特性，相当于将面向对象编程风格退化成了面向过程编程风格。

虽然我们将它们定义成private私有属性，但是提供了public的getter、setter方法，这就跟将这两个属性定义为public公有属性，没有什么两样了。外部可以通过setter方法随意地修改这两个属性的值。

面向对象封装的定义是：通过访问权限控制，隐藏内部数据，外部仅能通过类提供的有限的接口访问、修改内部数据。所以，暴露不应该暴露的setter方法，明显违反了面向对象的封装特性。数据没有访问权限控制，任何代码都可以随意修改它，代码就退化成了面向过程编程风格的了。

解决方法:设计实现类的时候，除非真的需要，否则，尽量不要给属性定义setter方法。

2.滥用常量类Constants和工具类Utils

常量是一种非常常见的全局变量，比如一些代码中的配置参数，一般都设置为常量，放到一个Constants类中。

我们常用的各种Utils类，里面的方法一般都会定义成静态方法，可以在不用创建对象的情况下，直接拿来使用。**静态方法将方法与数据分离，破坏了封装特性，是典型的面向过程风格。**

坏处如下：

- 这样的设计会影响代码的可维护性。如果参与开发同一个项目的工程师有很多，在开发过程中，可能都要涉及修改这个类，比如往这个类里添加常量，那这个类就会变得越来越大，成百上千行都有可能，查找修改某个常量也会变得比较费时，而且还会增加提交代码冲突的概率。
- 这样的设计还会增加代码的编译时间。当Constants类中包含很多常量定义的时候，依赖这个类的代码就会很多。那每次修改Constants类，都会导致依赖它的类文件重新编译，因此会浪费很多不必要的编译时间。
- 这样的设计还会影响代码的复用性。如果我们要在另一个项目中，复用本项目开发的某个类，而这个类又依赖Constants类。即便这个类只依赖Constants类中的一小部分常量，我们仍然需要把整个Constants类也一并引入，也就引入了很多无关的常量到新的项目中。

解决方法：

将Constants类拆解为功能更加单一的多个类，比如跟MySQL配置相关的常量，我们放到MysqlConstants类中；跟Redis配置相关的常量，我们放到RedisConstants类中。

只包含静态方法不包含任何属性的Utils类，是彻彻底底的面向过程的编程风格。但这并不是说，我们就要杜绝使用Utils类了。实际上，从刚刚讲的Utils类存在的目的来看，它在软件开发中还是挺有用的，能解决代码复用问题。所以，这里并不是说完全不能用Utils类，而是说，要尽量避免滥用，不要不加思考地随意去定义Utils类。

3.定义数据和方法分离的类

数据定义在一个类中，方法定义在另一个类中,这就是最常见的面向过程编程风格的代码。

MVC结构分为被分为Controller层、Service层、Repository层。Controller层负责暴露接口给前端调用，Service层负责核心业务逻辑，Repository层负责数据读写。而在每一层中，我们又会定义相应的VO (View Object)、BO (Business Object)、Entity。一般情况下，VO、BO、Entity中只会定义数据，不会定义方法，所有操作这些数据的业务逻辑都定义在对应的Controller类、Service类、Repository类中。这就是典型的面向过程的编程风格。

2.在面向对象编程中，为什么容易写出面向过程风格的代码？

- 在生活中，你去完成一个任务，你一般都会思考，应该先做什么、后做什么，如何一步一步地顺序执行一系列操作，最后完成整个任务。面向过程编程风格恰恰符合人的这种流程化思维方式。而面向对象编程风格正好相反。它是一种自底向上的思考方式。它不是先去按照执行流程来分解任务，而是将任务翻译成一个一个的小的模块（也就是类），设计类之间的交互，最后按照流程将类组装起来，完成整个任务。这样的思考路径比较适合复杂程序的开发，但并不是特别符合人类的思考习惯。
- 除此之外，面向对象编程要比面向过程编程难一些。在面向对象编程中，类的设计还是挺需要技巧，挺需要一定设计经验的。你要去思考如何封装合适的数据和方法到一个类里，如何设计类之间的关系，如何设计类之间的交互等等诸多设计问题。

所以，基于这两点原因，很多工程师在开发的过程，更倾向于用不太需要动脑子的方式去实现需求，也就不由自主地就将代码写成面向过程风格的了。

3.面向过程编程及面向过程编程语言就真的无用武之地了吗?

不管使用面向过程还是面向对象哪种风格来写代码，我们最终的目的还是写出易维护、易读、易复用、易扩展的高质量代码。只要我们能避免面向过程编程风格的一些弊端，控制好它的副作用，在掌控范围内为我们所用，我们就大可不用忌讳在面向对象编程中写面向过程风格的代码。

2.创建一个对象用什么运算符?对象实体与对象引用有何不同?

new 运算符，new 创建对象实例（对象实例在堆内存中），对象引用指向对象实例（对象引用存放在栈内存中）。

- 一个对象引用可以指向 0 个或 1 个对象（一根绳子可以不系气球，也可以系一个气球）；
- 一个对象可以有 n 个引用指向它（可以用 n 条绳子系住一个气球）

3.对象的相等和引用相等的区别

- 对象的相等(equals)一般比较的是内存中存放的内容是否相等。
- 引用相等(==)一般比较的是他们指向的内存地址是否相等。

```
String str1 = "hello";
String str2 = new String("hello");
String str3 = "hello";
// 使用 == 比较字符串的引用相等
System.out.println(str1 == str2);
System.out.println(str1 == str3);
// 使用 equals 方法比较字符串的相等
System.out.println(str1.equals(str2));
System.out.println(str1.equals(str3));
false
true
true
true
```

4.方法重载和重写的区别?

同个类中的多个方法可以有相同的方法名称，但是有不同的参数列表，这就称为方法重载。参数列表又叫参数签名，包括参数的类型、参数的个数、参数的顺序，只要有一个不同就叫做参数列表不同。

重载是面向对象的一个基本特性。

方法的重写描述的是父类和子类之间的。当父类的功能无法满足子类的需求，可以在子类对方法进行重写。方法重写时，方法名与形参列表必须一致。

如下代码，Person为父类，Student为子类，在Student中重写了dailyTask方法。

```
public class Person {
    private String name;

    public void dailyTask() {
        System.out.println("work eat sleep");
    }
}
```

```
public class Student extends Person {
    @Override
    public void dailyTask() {
        System.out.println("study eat sleep");
    }
}
```

5.构造方法有哪些特点？是否可被 override？

构造方法特点如下：

- 名字与类名相同。
- 没有返回值，但不能用 void 声明构造函数。
- 生成类的对象时自动执行，无需调用。

构造方法不能被 override（重写），但是可以 overload（重载），所以你可以看到一个类中有多个构造函数的情况。

```
public class MyClass {
    private int myVariable;

    public MyClass(int myVariable) {
        this.myVariable = myVariable;
    }
}
```

6.面向对象三大特性

面向对象三大特性：封装，继承，多态

1.封装

封装主要是用来隐藏信息和保护数据的。类通过暴露有限的访问接口，授权外部仅能通过类提供的方式（或者叫函数）来访问内部信息或者数据。

对于封装这个特性，我们需要编程语言本身提供一定的语法机制来支持。这个语法机制就是 **访问权限控制**。例子中的 private、public 等关键字就是 Java 语言中的访问权限控制语法。private 关键字修饰的属性只能类本身访问，可以保护其不被类之外的代码直接访问。如果 Java 语言没有提供访问权限控制语法，所有的属性默认都是 public 的，那任意外部代码都可以通过类似 wallet.id=123 这样的方式直接访问、修改属性，也就没办法达到隐藏信息和保护数据的目的了，也就无法支持封装特性了。

封装的意义是什么？它能解决什么编程问题？

- 如果我们对类中属性的访问不做限制，那任何代码都可以访问、修改类中的属性，虽然这样看起来更加灵活，但从另一方面来说，过度灵活也意味着不可控，属性可以随意被以各种奇葩的方式修改，而且修改逻辑可能散落在代码中的各个角落，势必影响代码的可读性、可维护性。
- **类仅仅通过有限的方法暴露必要的操作，也能提高类的易用性。如果我们把类属性都暴露给类的调用者，调用者想要正确地操作这些属性，就势必要对业务细节有足够的了解。而这对于调用者来说也是一种负担。相反，如果我们将属性封装起来，暴露少许的几个必要的方法给调用者使用，调用者就不需要了解太多背后的业务细节，用错的概率就减少很多**

2.继承

继承是从已有的类中派生出新的类，新的类继承父类的属性和行为，并能扩展新的能力，大大增加程序的重用性和易维护性。在Java中是单继承的，也就是说一个子类只有一个父类。

继承最大的一个好处就是代码复用。假如两个类有一些相同的属性和方法，我们就可以将这些相同的部分，抽取到父类中，让两个子类继承父类。这样，两个子类就可以重用父类中的代码，避免代码重复写多遍。

3.多态

多态是指在代码运行过程中可以用子类替代父类，并调用子类的方法。这样可以提高代码的复用性和灵活性。

举例:有一个动物类和他的2个子类猫和狗，他们都有一个名为makesound的方法，但是每种动物发出的声音是不同的。

```
class Animal{
    public void makesound(){

    }
}
class Cat extends Animal{
    @Override
    public void makesound() {
        System.out.println("喵");
    }
}
class Dog extends Animal{
    @Override
    public void makesound() {
        System.out.println("汪");
    }
}
public class Main{
    public static void main(String[] args) {
        Animal animal = new Cat();
        animal.makesound();
        animal = new Dog();
        animal.makesound();
    }
};
```

4.抽象

抽象讲的是如何隐藏方法的具体实现，让调用者只需要关心方法提供了哪些功能，并不需要知道这些功能是如何实现的。

7.接口和抽象类有什么共同点和区别？

接口和抽象类都是用于实现多态性的机制，它们在定义上有一些共同点，但也有一些关键的区别。具体分析如下：

- 共同点：

1. 两者都不能直接实例化，需要由具体的子类或实现类来进行实例化。
2. 都可以包含抽象方法，这些抽象方法需要由继承或实现它们的类来提供具体的实现。

- **不同点：**

1. **语法和声明方式：**接口使用 `interface` 关键字进行声明，而抽象类使用 `abstract` 关键字进行声明。
2. **成员变量：**接口只能定义公共静态最终字段（即常量），不能定义普通成员变量；而抽象类可以定义各种类型的成员变量。
3. **构造函数：**接口没有构造函数，而抽象类可以有构造函数。
4. **方法实现：**接口中的方法默认都是必须要实现的抽象方法，不包含具体的实现代码；而抽象类可以包含必须实现的抽象方法和可以直接继承的普通方法，其中抽象方法没有具体的实现，而普通方法有具体的实现代码。
5. **继承与实现：**一个类可以实现多个接口，但只能继承一个抽象类。
6. **访问修饰符：**抽象方法必须为 `public` 或者 `protected`，默认为 `public`；接口中的方法默认为 `public abstract`。
7. **静态成员：**接口中可以定义默认方法和静态方法（JDK 8及以上版本），而抽象类可以有静态代码块和静态方法。

总的来说，接口主要用于定义全局的规范，而抽象类则用于描述具有部分实现的共性。选择使用接口还是抽象类取决于具体的设计需求。

1.抽象类有什么用？

Java中引入抽象类的概念，可能是为了**实现接口和实现的分离、提高代码的复用性**、引导使用者正确使用以及设计上的灵活性等目的。以下是一些具体的原因：

1. **接口与实现分离：**抽象类允许开发者定义一个共同的接口（抽象方法），同时提供部分或全部的实现（非抽象方法）。这样，子类可以继承抽象类的通用行为，同时根据需要重写特定的方法。
2. **代码复用：**抽象类不能实例化，只能被继承。继承能解决代码复用的问题。所以，**抽象类也是为代码复用而生的。多个子类可以继承抽象类中定义的属性和方法，避免在子类中，重复编写相同的代码。**
3. **防止误用：**抽象类不能被实例化，这避免了直接使用不完整的基类，确保了必须使用某个具体的子类来实现完整的功能。这样的设计可以防止用户错误地使用不完整的类。

```
public abstract class AbstractClass {
    // 抽象方法，子类必须实现
    public abstract void abstractMethod();

    // 普通方法，子类可以直接使用或重写
    public void normalMethod() {
        System.out.println("This is a normal method in the abstract class.");
    }
}
```

2.接口有什么用？

接口与实现分离，为了解耦：接口里面有很多抽象方法，是对行为的一种抽象，相当于一组协议或者契约，你可以联想类比一下API接口。调用者只需要关注抽象的接口，不需要了解具体的实现，具体的实现代码对调用者透明。接口实现了方法和它具体实现相分离，可以降低代码间的耦合性，提高代码的可扩展性。

8.面向对象编程的六大原则？

- **对象单一职责**：我们设计创建的对象，必须职责明确，比如商品类，里面相关的属性和方法都必须跟商品相关，不能出现订单等不相关的内容。这里的类可以是模块、类库、程序集，而不单单指类。
- **里式替换原则**：子类能够完全替代父类，反之则不行。通常用于实现接口时运用。因为子类能够完全替代基（父）类，那么这样父类就拥有很多子类，在后续的程序扩展中就很容易进行扩展，程序完全不需要进行修改即可进行扩展。比如IA的实现为A，因为项目需求变更，现在需要新的实现，直接在容器注入处更换接口即可。
- **迪米特法则**，也叫最小原则，或者说最小耦合。通常在设计程序或开发程序的时候，尽量要高内聚，低耦合。当两个类进行交互的时候，会产生依赖。而迪米特法则就是建议这种依赖越少越好。就像构造函数注入父类对象时一样，当需要依赖某个对象时，并不在意其内部是怎么实现的，而是在容器中注入相应的实现，既符合里式替换原则，又起到了解耦的作用。
- **开闭原则**：开放扩展，封闭修改。当项目需求发生变更时，要尽可能的不去对原有的代码进行修改，而在原有的基础上进行扩展。
- **依赖倒置原则**：高层模块不应该直接依赖于底层模块的具体实现，而应该依赖于底层的抽象。接口和抽象类不应该依赖于实现类，而实现类依赖接口或抽象类。
- **接口隔离原则**：一个对象和另外一个对象交互的过程中，依赖的内容最小。也就是说在接口设计的时候，在遵循对象单一职责的情况下，尽量减少接口的内容

9.深拷贝和浅拷贝区别了解吗？什么是引用拷贝？

0.什么是深拷贝和浅拷贝？

浅拷贝

被复制对象的所有变量都含有与原来的对象相同的值，而所有的对其他对象的引用仍然指向原来的对象。即对象的浅拷贝会对“主”对象进行拷贝，但不会复制主对象里面的对象型变量。“里面的对象”会在原来的对象和它的副本之间共享。

简而言之，**浅拷贝仅仅复制所考虑的对象，而不复制它所引用的对象。**

深拷贝

深拷贝是一个整个独立的对象拷贝，深拷贝会拷贝所有的属性,并拷贝属性指向的动态分配的内存。当对象和它所引用的对象一起拷贝时即发生深拷贝。深拷贝相比于浅拷贝速度较慢并且花销较大。

简而言之，**深拷贝把要复制的对象所引用的对象都复制了一遍。**

1.浅拷贝示例代码

浅拷贝（Shallow Copy）：可以使用 `Object.clone()` 方法进行浅拷贝。需要注意的是，**要使一个类支持浅拷贝，必须实现 `Cloneable` 接口并重写 `clone()` 方法。**

```
class Address implements Cloneable{
    private String name;

    public Address(String s) {
        name = s;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
```

```

        this.name = name;
    }

    // 省略构造函数、Getter&Setter方法
    @Override
    public Address clone() {
        try {
            return (Address) super.clone();
        } catch (CloneNotSupportedException e) {
            throw new AssertionError();
        }
    }
}

class Person implements Cloneable {
    private Address address;

    public Person(Address address) {
        this.address = address;
    }

    public Address getAddress() {
        return address;
    }

    public void setAddress(Address address) {
        this.address = address;
    }

    // 省略构造函数、Getter&Setter方法
    @Override
    public Person clone() {
        try {
            Person person = (Person) super.clone();
            return person;
        } catch (CloneNotSupportedException e) {
            throw new AssertionError();
        }
    }
}

public class Test{
    public static void main(String[] args) {
        Person person1 = new Person(new Address("武汉"));
        Person person1Copy = person1.clone();
    // true
        System.out.println(person1.getAddress() == person1Copy.getAddress());

    }
}

```

//person1 的克隆对象和 person1 使用的仍然是同一个 Address 对象。

2.深拷贝示例代码

```
@Override
public Person clone() {
    try {
        Person person = (Person) super.clone();
        person.setAddress(person.getAddress().clone());
        return person;
    } catch (CloneNotSupportedException e) {
        throw new AssertionError();
    }
}

}

public class Test{
    public static void main(String[] args) {
        Person person1 = new Person(new Address("武汉"));
        Person person1Copy = person1.clone();
        // false
        System.out.println(person1.getAddress() == person1Copy.getAddress());

    }
}

// person1 的克隆对象和 person1 包含的 Address 对象已经是不同的了
```

1. JMM (Java 内存模型)

1.什么是JMM?

Java内存模型 (JMM) 是一种抽象的概念，描述了多线程环境下Java程序中共享变量的访问规则和原则。它确保了在不同线程之间共享数据时的一致性和可见性。

JMM内存模型由主内存和工作内存组成。

Java内存模型 (JMM) 定义了如何通过主内存和工作内存之间的交互来保证多线程环境下共享数据的一致性和可见性。每个线程都有自己的工作内存，这是私有的，而主内存则是所有线程共享的。在多线程环境中，JMM确保一个线程对共享变量的修改能够被其他线程及时看到，这就是所谓的“可见性”。同时，JMM还规定了一些操作必须按照某个顺序执行，这称为“有序性”。

2.主内存和工作内存

每个线程都有自己的工作内存，这是私有的，而主内存则是所有线程共享的。

- 主内存：所有的共享变量都存储于主内存(计算机的RAM)这里所说的变量指的是实例变量和类变量。不包含局部变量，因为局部变量是线程私有的，因此不存在竞争问题。
 - 当线程之间的数据需要同步时，必须通过主内存来完成。
- 工作内存：每个线程都有自己的工作内存（也称为本地内存），用于存储该线程使用的变量的副本
 - 线程对变量的所有的操作(读，写)都必须在工作内存中完成，而不能直接读写主内存中的变量，不同线程之间也不能直接访问对方工作内存中的变量，线程间变量的值的传递需要通过主内存完成。

3.JMM的关键特性

- 原子性
 - 原子性意味着一个或多个操作要么全部执行，要么全部不执行。在多线程环境下，确保数据操作的不可分割性，从而避免数据的不一致状态。例如，在并发访问共享资源时，原子操作可以防止其他线程干扰，确保操作的完整性。
- 可见性
 - 可见性确保当一个线程修改了共享变量的值后，这个修改对其他所有线程都是可见的。JMM通过规定线程在读写变量时必须与主内存进行交互来实现可见性。这样，当一个线程更新了共享变量后，其他线程读取该变量时将获得最新的值。
- 有序性
 - 有序性则确保了程序执行的顺序按照代码逻辑进行，但同时允许一些优化，如指令重排序，以提高性能。JMM规定了happens-before规则，即如果两个操作有happens-before关系，那么第一个操作的结果将对第二个操作可见，且第一个操作按顺序发生在第二个操作之前。

3.Java 内存区域和 JMM 有何区别？

这是一个比较常见的问题，很多初学者非常容易搞混。Java 内存区域和内存模型是完全不一样的两个东西：

- JVM 内存结构和 Java 虚拟机的运行时区域相关，定义了 JVM 在运行时如何分区存储程序数据，比如说堆主要用于存放对象实例。
- Java 内存模型和 Java 的并发编程相关，抽象了线程和主内存之间的关系比如说线程之间的共享变量必须存储在主内存中，规定了从 Java 源代码到 CPU 可执行指令的这个转

2.Java基础知识

-1. 一个Java文件里可以有多个类吗（不含内部类）？

1. 一个java文件里可以有多个类，但最多只能有一个被public修饰的类；
2. 如果这个java文件中包含public修饰的类，则这个类的名称必须和java文件名一致。

0.什么是值传递和引用传递？

- 值传递 (Pass by Value)：当传递基本数据类型（如int、float、double等）时，实际上是将变量的值传递给方法。这意味着在方法内部对参数进行的任何修改都不会影响到原始变量。
- 引用传递 (Pass by Reference)：当传递对象或数组时，实际上是将对象的引用（内存地址）传递给方法。这意味着在方法内部对参数进行的任何修改都会影响到原始对象。

代码举例

- 值传递

当我们将一个基本数据类型的变量作为参数传递给方法时，实际上是将该变量的值复制一份，然后传递给方法内的形参。这样，在方法内部对形参所做的任何修改都不会影响到原始变量。

在这个例子中，x 的值为 10。当我们调用 changeInt 方法时，我们传递了 x 的值给 y，而不是 x 变量本身。因此，在 changeInt 方法内部改变 y 的值不会影响到 main 方法中的 x。

```
public class ValuePassing {  
    public static void main(String[] args) {
```

```

        int x = 10;
        System.out.println("Before: " + x);
        changeInt(x);
        System.out.println("After: " + x);
    }

    public static void changeInt(int y) {
        y = 20;
    }
}
Before: 10
After: 10

```

- 引用传递

当我们将一个引用数据类型的变量（比如一个对象）作为参数传递给方法时，实际上传递的是这个对象引用的值。这意味着方法内部可以使用这个引用去访问和修改原始对象的状态。

```

public class ReferencePassing {
    public static void main(String[] args) {
        MyClass obj = new MyClass(10);
        System.out.println("Before: " + obj.getValue());
        changeObject(obj);
        System.out.println("After: " + obj.getValue());
    }

    public static void changeObject(MyClass obj) {
        obj.setValue(20);
    }
}

class MyClass {
    private int value;

    public MyClass(int value) {
        this.value = value;
    }

    public int getValue() {
        return value;
    }

    public void setValue(int value) {
        this.value = value;
    }
}

Before: 10
After: 20

```

1.c++和java的区别

C++和Java都是广泛使用的编程语言，但它们在多个方面存在显著差异。

1. **面向对象性**：Java是一种纯粹的面向对象编程语言，而C++既支持面向过程编程，也支持面向对象编程。在C++中，并非所有元素都必须是对象，你仍然可以有全局函数和全局变量。相比之下，Java中的所有代码（包括函数、变量）都必须在类中定义。

2. **内存管理**：C++允许程序员直接管理内存，包括使用指针和手动分配及释放内存。这提供了更高的控制能力，但也增加了出错的风险，如内存泄露和空悬指针等问题。相反，**Java自动进行垃圾回收（GC）**，负责监控并自动回收不再被引用的对象的内存空间，简化了内存管理，但也可能导致不可预测的GC暂停时间。
3. **跨平台能力**：Java的一个主要优点是其跨平台能力，Java代码在任何安装了Java虚拟机（JVM）的系统上都可以运行不变。C++代码则通常需要针对每个目标平台重新编译，虽然C++代码也可以跨平台，但需要更多的工作来适应不同的系统和编译器。
4. **执行速度**：C++通常提供比Java更快的执行速度，因为Java代码在运行时需要通过JVM转换为机器码，这一中间步骤会引入性能开销。C++代码直接编译成机器码，因此通常执行更快，但这也意味着C++程序更接近硬件层面，给开发者更多优化的空间。

2.Java的特点

Java是一门面向对象的编程语言。

Java具有平台独立性和移植性。

- Java有一句口号：`write once, run anywhere`，一次编写、到处运行。这也是Java的魅力所在。而实现这种特性的正是Java虚拟机JVM。已编译的Java程序可以在任何带有JVM的平台上运行。你可以在windows平台编写代码，然后拿到linux上运行。**只要你在编写完代码后，将代码编译成.class文件，再把class文件打成jar包，这个jar包就可以在不同的平台上运行了。**

Java具有稳健性。

- Java是一个强类型语言，它允许扩展编译时检查潜在类型不匹配问题的功能。Java要求显式的方法声明，它不支持C风格的隐式声明。这些严格的要求保证编译程序能捕捉调用错误，这就导致更可靠的程序。
- 异常处理是Java中使得程序更稳健的另一个特征。异常是某种类似于错误的异常条件出现的信号。使用 `try/catch/finally` 语句，程序员可以找到出错的处理代码，这就简化了出错处理和恢复的任务

3.Java是如何实现跨平台的？

JVM（Java虚拟机）是Java跨平台的关键。

在程序运行前，Java源代码（.java）需要经过编译器编译成字节码（.class）。在程序运行时，JVM负责将字节码翻译成特定平台下的机器码并运行，也就是说，**只要在不同的平台上安装对应的JVM，就可以运行字节码文件。**

同一份Java源代码在不同的平台上运行，它不需要做任何的改变，并且只需要编译一次。而编译好的字节码，是通过JVM这个中间的“桥梁”实现跨平台的，JVM是与平台相关的软件，它能将统一的字节码翻译成该平台的机器码

JVM（Java虚拟机）通过实现一次编写，到处运行的机制，使得Java程序可以在不同平台上运行。具体实现方式如下：

1. Java源代码：首先，将Java源代码编译成字节码文件（.class文件）。字节码是一种中间代码，介于源代码和机器码之间，具有平台无关性。
2. 字节码文件：字节码文件可以在任何安装了JVM的平台上运行。JVM负责将字节码文件解释执行或者即时编译成本地机器码。
3. JVM：JVM是Java程序的运行环境，它负责加载字节码文件、解释执行字节码或者将字节码即时编译成本地机器码。不同的操作系统和硬件平台上有不同的JVM实现，如Windows、Linux、macOS等。
4. 跨平台支持：由于JVM的存在，Java程序可以在不同的操作系统和硬件平台上运行，实现了一次编写，到处运行的目标。

4.为什么说 Java 语言“编译与解释并存”？

先看看什么是编译型语言和解释型语言。

我们可以将高级编程语言按照程序的执行方式分为两种：

- **编译型**：[编译型语言open in new window](#) 会通过[编译器open in new window](#)将源代码一次性翻译成可被该平台执行的机器码。一般情况下，编译语言的执行速度比较快，开发效率比较低。常见的编译性语言有 C、C++、Go、Rust 等等。
- **解释型**：[解释型语言open in new window](#)会通过[解释器open in new window](#)一句一句的将代码解释（interpret）为机器代码后再执行。解释型语言开发效率比较快，执行速度比较慢。常见的解释性语言有 Python、JavaScript、PHP 等等。

Java 语言既具有编译型语言的特征，也具有解释型语言的特征。因为 Java 程序要经过先编译，后解释两个步骤，由 Java 编写的程序需要先经过编译步骤，生成字节码（`.class` 文件），这种字节码必须由 Java 解释器来解释执行。

5.JDK/JRE/JVM三者的关系

JVM

英文名称（Java Virtual Machine），就是我们耳熟能详的 Java 虚拟机。Java 能够跨平台运行的核心在于 JVM。

所有的java程序会首先被编译为.class的类文件，这种类文件可以在虚拟机上执行。也就是说class文件并不直接与机器的操作系统交互，而是经过虚拟机间接与操作系统交互，由虚拟机将程序解释给本地系统执行。

针对不同的系统有不同的 jvm 实现，有 Linux 版本的 jvm 实现，也有Windows 版本的 jvm 实现，但是同一段代码在编译后的字节码是一样的。这就是Java能够跨平台，实现一次编写，多处运行的原因所在。

JRE

英文名称（Java Runtime Environment），就是Java 运行时环境。我们编写的Java程序必须要在JRE才能运行。它主要包含两个部分，JVM 和 Java 核心类库。

JRE是Java的运行环境，并不是一个开发环境，所以没有包含任何开发工具，如编译器和调试器等。

如果你只是想运行Java程序，而不是开发Java程序的话，那么你只需要安装JRE即可。

JDK

英文名称（Java Development Kit），就是 Java 开发工具包

可以看到，JDK目录下有个JRE，也就是JDK中已经集成了 JRE，不用单独安装JRE。

另外，JDK中还有一些好用的工具，如jinfo, jps, jstack等。

最后，总结一下JDK/JRE/JVM，他们三者的关系

JRE = JVM + Java 核心类库

JDK = JRE + Java工具 + 编译器 + 调试器

6.Java创建对象有几种方式？

Java创建对象有以下几种方式：

- 用new语句创建对象。
- 使用反射，使用Class.newInstance()创建对象。

- 调用对象的clone()方法。
- 运用反序列化手段，调用java.io.ObjectInputStream对象的readObject()方法。

7.说说类实例化的顺序

Java中类实例化顺序：

1. 静态属性，静态代码块。
2. 普通属性，普通代码块。
3. 构造方法。

8.Java的4种引用类型(见jvm文章)

在java1.2之后，java对引用的概念进行了扩充，将引用分为强引用、软引用、弱引用、虚引用，这4种引用强度依次逐渐减弱。

- 强引用（Strong Reference）：在程序代码之中普遍存在的，类似 `Object obj = new Object()` 这类的引用，**只要强引用还存在，垃圾收集器永远不会回收掉被引用的对象**
- 软引用（Soft Reference）：用来描述一些还有用但并非必需的对象。对于软引用关联着的对象，**在系统将要发生内存溢出异常之前，将会把这些对象列进回收范围之中进行第二次回收。如果这次回收还没有足够的内存才会抛出内存溢出异常**
- 弱引用（Weak Reference）：用来描述非必需对象，但是它的强度比软引用更弱一些，**被弱引用关联的对象只能生存到下一次垃圾收集发生之前。当垃圾收集器工作时，无论当前内存是否足够，都会回收掉只被弱引用关联的对象**
- 虚引用（Phantom Reference）：它是最弱的一种引用关系。**一个对象是否有虚引用存在，完全不会对其生存时间构成影响，也无法通过虚引用来取得一个对象实例。为一个对象设置虚引用关联的唯一目的就是能在这个对象被收集器回收时收到一个系统通知**

10.简述 Java 访问修饰符

- default: 默认访问修饰符，在同一包内可见
- private: 在同一类内可见，不能修饰类
- protected : 对同一包内的类和所有子类可见，不能修饰类
- public: 对所有类可见

11.构造方法、成员变量初始化以及静态成员变量三者的初始化顺序？

先后顺序：静态成员变量、成员变量、构造方法。

详细的先后顺序：父类静态变量、父类静态代码块、子类静态变量、子类静态代码块、父类非静态变量、父类非静态代码块、父类构造函数、子类非静态变量、子类非静态代码块、子类构造函数。

12.Java 代码块执行顺序

- 父类静态代码块（只执行一次）
- 子类静态代码块（只执行一次）
- 父类构造代码块
- 父类构造函数
- 子类构造代码块
- 子类构造函数
- 普通代码块

13.为什么 Java 语言不支持多重继承？

为了程序的结构能够更加清晰从而便于维护。假设 Java 语言支持多重继承，类 C 继承自类 A 和类 B，如果类 A 和 B 都有自定义的成员方法 `f()`，那么当代码中调用类 C 的 `f()` 会产生二义性。

Java 语言通过实现多个接口间接支持多重继承，接口由于只包含方法定义，不能有方法的实现，类 C 继承接口 A 与接口 B 时即使它们都有方法 `f()`，也不能直接调用方法，需实现具体的 `f()` 方法才能调用，不会产生二义性。

多重继承会使类型转换、构造方法的调用顺序变得复杂，会影响到性能。

3.Java数据类型

0.java数据类型有哪些？

有基本数据类型和引用数据类型

在Java中，引用类型主要包括类（Class）、接口（Interface）、数组（Array）以及基于这些结构的其他数据类型如字符串（String）和枚举类型（Enum）。

Java将数据类型主要分为两大类：基本数据类型和引用数据类型。基本数据类型包括byte、short、int、long、float、double、char和boolean，它们直接存储值而非对象的引用。引用类型的变量则存储的是对象在内存中的地址，即引用了对象的位置。

final关键字修饰的成员变量如果是引用类型的话，则表示这个引用的地址值是不能改变的，但是这个引用所指向的对象里面的内容还是可以改变的。

而集合类都是引用类型，用final修饰的话，集合里面的内容还是可以修改的。

1.Java的基本数据类型有哪些？ 每个占多少位？

- byte：1字节（8位），数据范围是 $-2^7 \sim 2^7-1$ 。
- short：2字节（16位），数据范围是 $-2^{15} \sim 2^{15}-1$ 。
- int：4字节（32位），数据范围是 $-2^{31} \sim 2^{31}-1$ 。
- float：4字节（32位），数据范围大约是 $-3.410^{38} \sim 3.410^{38}$ 。
- long：8字节（64位），数据范围是 $-2^{63} \sim 2^{63}-1$ 。
- double：8字节（64位），数据范围大约是 $-1.810^{308} \sim 1.810^{308}$ 。
- char：2字节（16位），数据范围是 `\u0000 ~ \uffff`。
- boolean：1位

简单类型	boolean	byte	char	short	Int	long	float	double
二进制位数	1	8	16	16	32	64	32	64
包装类	Boolean	Byte	Character	Short	Integer	Long	Float	Double

在Java规范中，没有明确指出boolean的大小。在《Java虚拟机规范》给出了单个boolean占4个字节，和boolean数组1个字节的定义，具体 **还要看虚拟机实现是否按照规范来**，因此boolean占用1个字节或者4个字节都是有可能的。

2.int类型的数据范围是多少？

int类型占4字节（32位），数据范围是 $-2^{31} \sim 2^{31}-1$ 。

2.了解Java的包装类型吗？为什么需要包装类？

Java包装类型有8种，分别是：1、Byte；2、Integer；3、Short；4、Long；5、Float；6、Double；7、Boolean；8、Character。

Java 是一种面向对象语言，很多地方都需要使用对象而不是基本数据类型。比如，在集合类中，我们是无法将 int、double 等类型放进去的。因为集合的容器要求元素是 Object 类型。

```
ArrayList<Integer> numbers = new ArrayList<>();
```

为了让基本类型也具有对象的特征，就出现了包装类型。相当于将基本类型包装起来，使得它具有了对象的性质，并且为其添加了属性和方法，丰富了基本类型的操作。

3.自动装箱与拆箱了解吗？原理是什么？

什么是自动拆装箱？

- 自动装箱：可以把一个基本类型的数据直接赋值给对应的包装类型；
- 自动拆箱：可以把一个包装类型的对象直接赋值给对应的基本类型；

通过自动装箱、自动拆箱功能，可以大大简化基本类型变量和包装类对象之间的转换过程。比如，某个方法的参数类型为包装类型，调用时我们所持有的数据却是基本类型的值，则可以不做任何特殊的处理，直接将这个基本类型的值传入给方法即可。

原理方面，自动装箱时编译器会调用包装类的 `valueOf` 方法将原始类型的值转换成对象。而在自动拆箱的过程中，编译器则通过调用类似 `intValue()`、`doubleValue()` 这样的方法将对象转换回原始类型的值。

示例代码：

```
Integer x = 1; // 装箱 调用 Integer.valueOf(1)
int y = x; // 拆箱 调用了 x.intValue()
```

4.为什么浮点数运算的时候会有精度丢失的风险？

为什么会出现这个问题呢？

这个和计算机保存浮点数的机制有很大关系。我们知道计算机是二进制的，而且计算机在表示一个数字时，宽度是有限的，无限循环的小数存储在计算机时，只能被截断，所以就会导致小数精度发生损失的情况。这也就是解释了为什么浮点数没有办法用二进制精确表示。

1.如何解决浮点数运算的精度丢失问题？

`BigDecimal` 可以实现对浮点数的运算，不会造成精度丢失。通常情况下，大部分需要浮点数精确运算结果的业务场景（比如涉及到钱的场景）都是通过 `BigDecimal` 来做的。

```
import java.math.BigDecimal;
```



```

public class BigDecimalDemo {
    public static void main(String[] args) {
        BigDecimal a = new BigDecimal("0.1");
        BigDecimal b = new BigDecimal("0.2");
        BigDecimal c = a.add(b); // 加法运算
        System.out.println("a + b = " + c);

        BigDecimal d = new BigDecimal("0.3");
        BigDecimal e = new BigDecimal("0.4");
        BigDecimal f = d.subtract(e); // 减法运算
        System.out.println("d - e = " + f);

        BigDecimal g = new BigDecimal("0.5");
        BigDecimal h = new BigDecimal("0.6");
        BigDecimal i = g.multiply(h); // 乘法运算
        System.out.println("g * h = " + i);

        BigDecimal j = new BigDecimal("0.7");
        BigDecimal k = new BigDecimal("0.8");
        BigDecimal l = j.divide(k, 2, BigDecimal.ROUND_HALF_UP); // 除法运算，保留
        // 两位小数，四舍五入
        System.out.println("j / k = " + l);
    }
}

```

2.BigDecimal的底层原理

- 数值首先被扩大到一个没有小数点的整数值。记录这时候的扩大的倍数
- 在BigDecimal内部，它使用一个整数数组来存储扩大后的数值的每一位，这个数组的每一个元素代表一组十进制数的位数。例如，数组的第一个元素表示最低位，第二个元素表示十位，以此类推。这种设计使得BigDecimal在进行加法、减法、乘法和除法等算术运算时，可以逐位进行计算，从而保持了高精度的结果。
- 直到最终结果需要被使用时才根据需要的精度进行缩放

5.包装类型的缓存机制了解么？

Byte, Short, Integer, Long 这 4 种包装类默认创建了数值 **[-128, 127]** 的相应类型的缓存数据，Character 创建了数值在 **[0,127]** 范围的缓存数据，Boolean 直接返回 True or False。

如果超出对应范围仍然会去创建新的对象，缓存的范围区间的大小只是在性能和资源之间的权衡。

举例1

```
Integer a = 100;//自动装箱
Integer b = 100;
System.out.println(a == b);//true

Integer c = 200;
Integer d = 200;
System.out.println(c == d);//false
```

这是因为Java虚拟机 (JVM) 对 `Integer` 对象进行了缓存优化。对于在-128到127之间的整数, JVM会缓存这些对象的实例, 以便重复使用。因此, 当我们声明 `a` 和 `b` 并赋值为100时, 它们都指向相同的缓存对象, 所以 `a == b` 的结果为 `true`。

然而, 对于超出-128到127范围的整数, JVM不会进行缓存优化。因此, 当我们声明 `c` 和 `d` 并赋值为200时, 它们分别创建了两个不同的对象, 所以 `c == d` 的结果为 `false`。

举例2

下面我们来看一个问题: 下面的代码的输出结果是 `true` 还是 `false` 呢?

```
Integer i1 = 40;
Integer i2 = new Integer(40);
System.out.println(i1==i2);
//false
```

`Integer i1=40` 这一行代码会发生装箱, 也就是说这行代码等价于 `Integer i1=Integer.valueOf(40)`。因此, `i1` 直接使用缓存中的对象。而 `Integer i2 = new Integer(40)` 会直接创建新的对象。

6.如何对Integer和Double类型判断相等?

`Integer`、`Double`不能直接进行比较, 这包括:

不能用`==`进行直接比较, 因为它们是不同的数据类型;

不能转为字符串进行比较, 因为转为字符串后, 浮点值带小数点, 整数值不带, 这样它们永远都不相等;

不能使用`compareTo`方法进行比较, 虽然它们都有`compareTo`方法, 但该方法只能对相同类型进行比较。

整数、浮点类型的包装类, 都继承于`Number`类型, 而`Number`类型分别定义了将数字转换为`byte`、`short`、`int`、`long`、`float`、`double`的方法。所以, 可以将`Integer`、`Double`先转为转换为相同的基本数据类型 (如`double`), 然后使用`==`进行比较。

```
Integer i = 100;
Double d = 100.00;
System.out.println(i.doubleValue() == d.doubleValue());
```

7.11 int和Integer有什么区别, 二者在做==运算时会得到什么结果?

`int`是基本数据类型, `Integer`是`int`的包装类。二者在做`==`运算时, `Integer`会自动拆箱为`int`类型, 然后再进行比较。届时, 如果两个`int`值相等则返回`true`, 否则就返回`false`。

4.Object

1.Object常用方法有哪些？

见博客:[Object常用方法 - 掘金\(juejin.cn\)](http://juejin.cn)

Object常用方法有：`toString()`、`equals()`、`hashCode()`、`clone()` 等。

- **toString**:默认输出对象地址。可以重写toString方法，按照重写逻辑输出对象值。
- **equals**

默认比较两个引用变量是否指向同一个对象（内存地址）。可以重写equals方法，按照属性是否相等来判断

- **hashCode**

将与对象相关的信息映射成一个哈希值，默认的实现hashCode值是根据内存地址换算出来。

- **clone**

Java赋值是复制对象引用，如果我们想要得到一个对象的副本，使用赋值操作是无法达到目的的。

Object对象有个clone()方法，实现了对象中各个属性的复制，但它的可见范围是protected的。

```
protected native Object clone() throws CloneNotSupportedException;
```

所以实体类使用克隆的前提是：

- 实现Cloneable接口，这是一个标记接口，自身没有方法，这应该是一种约定。调用clone方法时，会判断有没有实现Cloneable接口，没有实现Cloneable的话会抛异常CloneNotSupportedException。
- 覆盖clone()方法，可见性提升为public。

getClass

返回此 Object 的运行时常类，常用于java反射机制。

2.equals和引用相等==的区别

- **==运算符**:
 - 作用于基本数据类型时，是比较两个数值是否相等；
 - 作用于引用数据类型时，是比较两个对象的内存地址是否相同，即判断它们是否为同一个对象；
- **equals()方法**:
 - 没有重写时，Object默认以 == 来实现，即比较两个对象的内存地址是否相同；
 - 进行重写后，一般会按照对象的内容来进行比较，若两个对象内容相同则认为对象相等，否则认为对象不等。

```
String str1 = "hello";// 放在常量池中
String str2 = new String("hello");
String str3 = "hello";// 放在常量池中
// 使用 == 比较字符串的引用相等
System.out.println(str1 == str2);
System.out.println(str1 == str3);
// 使用 equals 方法比较字符串的相等
System.out.println(str1.equals(str2));
System.out.println(str1.equals(str3));
false
true
true
true
```

`String` 中的 `equals` 方法是被重写过的，因为 `Object` 的 `equals` 方法是比较的对象的内存地址，而 `String` 的 `equals` 方法比较的是对象的值。

3.hashCode() 有什么用？

`hashCode()` 的作用是获取哈希码（`int` 整数），也称为散列码。这个哈希码的作用是确定该对象在哈希表中的索引位置。

4.为什么要有 hashCode？

- **快速查找：**在哈希表中，`hashCode` 被用于计算存储或检索元素的索引。这使得查找、插入和删除操作可以在平均时间复杂度为 $O(1)$ 的情况下完成，即几乎恒定的时间。
- **提升对象比较的效率：**当需要判断两个对象是否相等时，首先会比较它们的 `hashCode`。如果 `hashCode` 不同，则可以立即断定这两个对象不相等，从而避免了可能更耗时的对象比较。
 - 我们以“`HashSet` 如何检查重复”为例子来说明为什么要有 `hashCode`？
当你把对象加入 `HashSet` 时，`HashSet` 会先计算对象的 `hashCode` 值来判断对象加入的位置，同时也会与其他已经加入的对象的 `hashCode` 值作比较，如果没有相符的 `hashCode`，`HashSet` 会假设对象没有重复出现。但是**如果发现有相同 `hashCode` 值的对象，这时会调用 `equals()` 方法来检查 `hashCode` 相等的对象是否真的相同**。如果两者相同，`HashSet` 就不会让其加入操作成功。如果不同的话，就会重新散列到其他位置。这样我们就**大大减少了 `equals` 的次数，相应就大大提高了执行速度**
 - 其实，`hashCode()` 和 `equals()` 都是用于比较两个对象是否相等。
- **一致性：**对于相同的对象，`hashCode` 必须始终返回相同的值，只要对象的状态没有改变。这保证了哈希表的一致性，即使多次调用 `hashCode` 方法，也能得到相同的索引位置。

5.为什么重写 equals 时一定要重写 hashCode？

`Object`类提供的`equals()`方法默认是用 `==` 来进行比较的，也就是说只有两个对象是同一个对象时，才能

返回相等的结果。而实际的业务中，我们通常的需求是，若两个不同的对象它们的内容是相同的，就认为它们相等。鉴于这种情况，`Object`类中`equals()`方法的默认实现是没有实用价值的，所以通常都要重写。

`hashCode()`用于获取哈希码（散列码），`eauqls()`用于比较两个对象是否相等，它们应遵守如下规定：

- 如果两个对象相等，则它们必须有相同的哈希码。
- 如果两个对象有相同的哈希码，则它们未必相等(哈希冲突)

因为两个相等的对象的 `hashCode` 值必须是相等。也就是说如果 `equals` 方法判断两个对象是相等的，那这两个对象的 `hashCode` 值也要相等。

如果重写 `equals()` 时没有重写 `hashCode()` 方法的话就可能会导致 `equals` 方法判断是相等的两个对象，`hashCode` 值却不相等。这样，当用其中的一个对象作为键保存到`HashMap`、`HashTable`或`HashSet`中，再以另一个对象作为键值去查找他们的时候，则会查找不到。

重写equals()的例子

在这个示例中，我们创建了一个 `Person` 类，包含 `name` 和 `age` 两个属性。我们重写了 `hashCode()` 方法，首先定义一个初始值为17的变量 `result`，然后分别将 `name` 和 `age` 的哈希值乘以31并加到 `result` 上，最后返回 `result` 作为对象的哈希码值。这样做的目的是为了让不同的对象尽可能地产生不同的哈希码值，从而提高哈希表等数据结构的性能。

```

public class Person {
    private String name;
    private int age;

    // 构造函数和其他方法...

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null || getClass() != obj.getClass()) {
            return false;
        }
        Person person = (Person) obj;
        return age == person.age && name.equals(person.name);
    }

    @Override
    public int hashCode() {
        int result = 17; // 初始值
        result = 31 * result + name.hashCode(); // 使用字符串的hashCode()方法计算哈希
        值
        result = 31 * result + age; // 直接将年龄加入哈希值计算
        return result;
    }
}

```

6.两个对象的hashCode()相同，则 equals()是否也一定为true?

- equals 方法判断两个对象是相等的，那这两个对象的 hashCode 值也要相等。
- 两个对象有相同的 hashCode 值，他们也不一定是相等的（哈希碰撞）。

hashCode方法主要是用来**提升对象比较的效率**，先进行hashCode()的比较，如果不相同，那就不必在进行equals的比较，这样就大大减少了equals比较的次数，当比较对象的数量很大的时候能提升效率。

5.String

-1.String类有哪些方法?

- indexOf(): 返回指定字符的索引。
- charAt(): 返回指定索引处的字符。
- replace(): 字符串替换。

- trim(): 去除字符串两端空白。
- split(): 分割字符串, 返回一个分割后的字符串数组。
- getBytes(): 返回字符串的 byte 类型数组。
- length(): 返回字符串长度。
- toLowerCase(): 将字符串转成小写字母。
- toUpperCase(): 将字符串转成大写字母。
- substring(): 截取字符串。
- equals(): 字符串比较。

0.String可以被继承吗?

String类由final修饰, 所以不能被继承。

0.new String 与String.valueOf()的区别?

在Java中, `new String()` 和 `String.valueOf()` 都可以用来创建字符串, 但它们之间存在一些差异。具体来说:

1. **内存分配:** 使用 `new String("a")` 每次都会在堆内存中创建一个新的对象, 即使内容相同。而使用 `String.valueOf("a")` 时, 如果值已经存在于字符串常量池中, 则直接引用该值, 不会额外分配内存。
2. **性能考虑:** 由于 `new String("a")` 会频繁创建相同的字符串对象, 可能会导致内存浪费和性能下降。相比之下, `String.valueOf()` 通常更加高效, 因为它可能会重用已有的字符串实例。
3. **空指针异常:** 当传入null作为参数时, `new String(null)` 将抛出一个 `NullPointerException`, 而 `String.valueOf(null)` 会返回字符串"null", 不会导致异常。
4. **类型转换:** `String.valueOf()` 是一个静态方法, 它可以将各种数据类型转换为其字符串表示, 包括基本类型和对象类型。这使得它在需要将不同类型的数据转换为字符串时非常有用。

总的来说, `new String()` 和 `String.valueOf()` 都可以用来创建字符串, 但它们在内存分配、性能考虑以及空指针异常等方面有所区别。在实际开发中, 推荐使用 `String.valueOf()`, 因为它更加灵活且性能更好。

1.String, StringBuffer 和 StringBuilder区别

- String
 - String 类表示不可变的字符序列。一旦一个 String 对象被创建, 它的值就不能改变。这意味着每次你对一个 String 进行修改 (如拼接), 都会创建一个新的 String 对象, 这可能导致额外的内存消耗和性能开销。
 - 线程安全
 - ```
String s1 = "Hello";
s1 += " world"; // 这将创建一个新的 String 对象
```
- StringBuffer
  - **StringBuffer 和 StringBuilder 可变,指修改值不会创建新的对象**
  - StringBuffer 类类似于 StringBuilder, 但是它是**线程安全**的。这意味着多个线程可以安全地共享一个 StringBuffer 实例而不会导致数据不一致的问题。然而, 由于线程安全需要加锁机制, 因此 StringBuffer 在单线程环境下通常比 StringBuilder 慢。
  - ```
StringBuffer sb = new StringBuffer("Hello");  
sb.append(" world"); // 不会创建新的对象
```

- `StringBuilder`
 - `StringBuilder` 类类似于 `StringBuffer`，但它是非线程安全的。这意味着它在单线程环境下提供了更好的性能，因为它不需要进行线程同步。在多线程环境中使用 `StringBuilder` 可能会导致数据不一致问题。
 - ```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" world"); // 不会创建新的对象
```

总结：

- 如果你需要处理不可变的字符串，使用 `String`。
- 如果你需要在多线程环境下处理可变的字符串，使用 `StringBuffer`。
- 如果你确信你的代码只在一个线程中运行，并且需要处理大量字符串操作，使用 `StringBuilder` 以获得更好的性能。

## 2.String 为什么不可变？

先看看什么是不可变的对象。

如果一个对象，在它创建完成之后，不能再改变它的状态，那么这个对象就是不可变的。不能改变状态的意思是，不能改变对象内的成员变量，包括基本数据类型的值不能改变，引用类型的变量不能指向其他的对象，引用类型指向的对象的狀態也不能改变。

接着来看Java8 String类的源码：

```
public final class String
 implements java.io.Serializable, Comparable<String>, CharSequence {
 /** The value is used for character storage. */
 private final char value[];

 /** Cache the hash code for the string */
 private int hash; // Default to 0
}
```

从源码可以看出，String对象其实在内部就是一个个字符，存储在这个value数组里面的。

(1)value数组用final修饰，final 修饰的变量，值不能被修改。因此value不可以指向其他对象。

(2)String类内部所有的字段都是私有的，也就是被private修饰。而且String没有对外提供修改内部状态的方法，因此value数组不能改变。

所以，String是不可变的。

## 为什么String要设计成不可变的？

主要有以下几点原因：

1. **线程安全**。同一个字符串实例可以被多个线程共享，因为字符串不可变，本身就是线程安全的。
2. **支持hash映射和缓存**。因为String的hash值经常会使用到，比如作为 Map 的键，不可变的特性使得 hash 值也不会变，不需要重新计算。
3. **出于安全考虑**。网络地址URL、文件路径path、密码通常情况下都是以String类型保存，假若String不是固定不变的，将会引起各种安全隐患。比如将密码用String的类型保存，那么它将一直留在内存中，直到垃圾收集器把它清除。假如String类不是固定不变的，那么这个密码可能会被改变，导致出现安全隐患。
4. **字符串常量池优化**。String对象创建之后，会缓存到字符串常量池中，下次需要创建同样的对象时，可以直接返回缓存的引用。



既然我们的String是不可变的，它内部还有很多substring，replace，replaceAll这些操作的方法。这些方法好像会改变String对象？怎么解释呢？

其实不是的，我们每次调用replace等方法，其实会在堆内存中创建了一个新的对象。然后其value数组引用指向不同的对象。

### 3.String.equals() 和 Object.equals() 有何区别？

String 中的 equals 方法是被重写过的，比较的是 String 字符串的值是否相等。Object 的 equals 方法是比较的对象的内存地址。

### 3.使用字符串时，new String()和""直接量有什么区别？

先看看 "hello" 和 new String("hello") 的区别：

- 当Java程序直接使用 "hello" 的字符串直接量时，
  - JVM会先检查常量池中是否已经存有"abc"，若没有则将"abc"存入常量池，否则就复用常量池中已有的"abc"，将其引用赋值给变量a。
- 当使用 new String("hello") 时，JVM会先在字符串常量池里看有没有这个字符串，若有则在堆内存上创建个String对象；若没有，创建此字符串并放入池中，再在堆内存上创建各String对象。

显然，采用new的方式会多创建一个对象出来，会占用更多的内存，所以一般建议使用直接量的方式创建字符串

### 4.什么是字符串常量池？

**字符串常量池** (String Pool)是 JVM 为了提升性能和减少内存消耗针对字符串（String 类）专门开辟的一块区域，主要目的是**为了避免字符串的重复创建**。在创建字符串时，JVM首先会检查字符串常量池，如果该字符串已经存在池中，则返回其引用，如果不存在，则创建此字符串并放入池中，并返回其引用。

```
// 将字符串对象"ab"的引用保存在字符串常量池中
String aa = "ab";
// 直接返回字符串常量池中字符串对象"ab"的引用
String bb = "ab";
System.out.println(aa==bb);// true
```

### 4.String a = "abc";，说一下这个过程会创建什么，放在哪里？

JVM会使用常量池来管理字符串直接量。在执行这句话时，JVM会先检查常量池中是否已经存有"abc"，若没有则将"abc"存入常量池，否则就复用常量池中已有的"abc"，将其引用赋值给变量a。

### 5.String s = new String("abc")会创建几个对象？

new String()无论如何都会先

会创建 1 或 2 个字符串对象。

1、如果字符串常量池中不存在字符串对象“abc”的引用，那么它将首先在字符串常量池中创建，然后在堆空间中创建，因此将创建总共 2 个字符串对象。

示例代码（JDK 1.8）：

```
String s1 = new String("abc");
```

2、如果字符串常量池中已存在字符串对象“abc”的引用，则只会在堆中创建 1 个字符串对象“abc”。

示例代码（JDK 1.8）：

```
// 字符串常量池中已存在字符串对象“abc”的引用
String s1 = "abc";
// 下面这段代码只会在堆中创建 1 个字符串对象“abc”
String s2 = new String("abc");
```

## 7.为何JDK9要将String的底层实现由char[]改成byte[]?

主要是为了节约String占用的内存。

**char占2字节，byte占1字节**

在大部分Java程序的堆内存中，String占用的空间最大，并且绝大多数String只有Latin-1字符，这些Latin-1字符只需要1个字节就够了。

而在JDK9之前，JVM因为String使用char数组存储，每个char占2个字节，所以即使字符串只需要1字节，它也要按照2字节进行分配，浪费了一半的内存空间。

到了JDK9之后，对于每个字符串，会先判断它是不是只有Latin-1字符，如果是，就按照1字节的规格进行分配内存，如果不是，就按照2字节的规格进行分配，这样便提高了内存使用率，同时GC次数也会减少，提升效率。

不过Latin-1编码集支持的字符有限，比如不支持中文字符，因此对于中文字符串，用的是UTF16编码（两个字节），所以用byte[]和char[]实现没什么区别。

## 6.异常

### 0.常见的异常有哪些？

- NullPointerException 空指针异常
- ArrayIndexOutOfBoundsException 索引越界异常
- InputFormatException 输入类型不匹配
- SQLException SQL异常
- IllegalArgumentException 非法参数
- NumberFormatException 类型转换异常

### 1.Exception 和 Error 有什么区别？

在Java中，所有的异常都有一个共同的祖先 `java.lang` 包中的 `Throwable` 类。`Throwable` 类有两个重要的子类：

- **Exception**：程序本身可以处理的异常，可以通过 `catch` 来进行捕获。`Exception` 又可以分为 `Checked Exception` (受检查异常，必须处理) 和 `Runtime Exception` (运行时异常，可以不处理)。
- **Error**：`Error` 属于程序无法处理的错误，我们没办法通过 `catch` 来进行捕获不建议通过 `catch` 捕获。例如Java虚拟机运行错误（`Virtual MachineError`）、虚拟机内存不够错误（`OutOfMemoryError`）、类定义错误（`NoClassDefFoundError`）等。这些异常发生时，Java虚拟机（JVM）一般会选择线程终止。

- 一般是指与虚拟机相关的问题，如系统崩溃、虚拟机错误、动态链接失败等，这种错误无法恢复或不可能捕获，将导致应用程序中断。通常应用程序无法处理这些错误，因此应用程序不应该试图使用catch块来捕获Error对象。

## 2.运行时异常(RuntimeException)和非运行时异常(chcheckedExecption) 的区别?

unchecked exception 包括 RuntimeException 和 Error 类，其他所有异常称为检查（checked）异常。

1. RuntimeException 由程序错误导致，应该修正程序避免这类异常发生。Java 代码在编译过程中，我们即使不处理不受检查异常也可以正常通过编译。
2. checked Exception 由具体的环境（读取的文件不存在或文件为空或sql异常）导致的异常。必须进行处理，不然编译不通过，可以catch或者throws。

## 3.异常要怎么解决?

Java标准库内建了一些通用的异常，这些类以Throwable为顶层父类。

Throwable又派生出Error类和Exception类。

错误：Error类以及他的子类的实例，代表了JVM本身的错误。错误不能被程序员通过代码处理，Error很少出现。因此，程序员应该关注Exception为父类的分支下的各种异常类。

异常：Exception以及他的子类，代表程序运行时发送的各种不希望发生的事件。可以被Java异常处理机制使用，是异常处理的核心。

处理方法:

- 1.try()catch(){}
  - try 关键字作用是抛出一个异常，抛出的时候是抛出的的是一个异常类的实例化对象，在异常处理中，try 语句要捕获的是一个异常对象，那么此异常对象也可以自己抛出
- 2.throw
  - throw 关键字作用是抛出一个异常，抛出的时候是抛出的的是一个异常类的实例化对象，在异常处理中，try 语句要捕获的是一个异常对象，那么此异常对象也可以自己抛出
- 3.throws
  - 定义一个方法的时候可以使用 throws 关键字声明。使用 throws 关键字声明的方法表示此方法不处理异常，而交给方法调用处进行处理。

## 3.Throwable 类常用方法有哪些?

- String getMessage(): 返回异常发生时的简要描述
- String toString(): 返回异常发生时的详细信息
- String getLocalizedMessage(): 返回异常对象的本地化信息。使用 Throwable 的子类覆盖这个方法，可以生成本地化信息。如果子类没有覆盖该方法，则该方法返回的信息与 getMessage() 返回的结果相同
- void printStackTrace(): 在控制台上打印 Throwable 对象封装的异常信息

## 4.try-catch-finally 如何使用?

- try 块：用于捕获异常。其后可接零个或多个 catch 块，如果没有 catch 块，则必须跟一个 finally 块。
- catch 块：用于处理 try 捕获到的异常。
- finally 块：无论是否捕获或处理异常，finally 块里的语句都会被执行。当在 try 块或 catch 块中遇到 return 语句时，finally 语句块将在方法返回之前被执行。

## 5.Java中的finally一定会被执行吗？

答案是不一定。

有以下两种情况finally不会被执行：

- 程序未执行到try代码块
- 当程序使用System.exit(int)退出
- 如果当一个线程在执行 try 语句块或者 catch 语句块时被打断（interrupted）或者被终止（killed），与其相对应的 finally 语句块可能不会执行。还有更极端的情况，就是在线程运行 try 语句块或者 catch 语句块时，突然死机或者断电，finally 语句块肯定不会执行了。
- **try里break和return都不会影响finally执行**

```
○ try{
 return;
}catch (Exception e){
 e.printStackTrace();
}finally {
 System.out.println("1");
}
```

## 6.throw和throws的区别？

- (1) throws用于方法头，表示的只是异常的申明，而throw用于方法内部，抛出的是异常对象
- (2) throws可以一次性抛出多个异常，而throw只能一个
- (3) throws抛出异常时，它的上级（调用者）也要申明抛出异常或者捕获，不然编译报错。而throw的话，可以不申明或不捕获（这是非常不负责任的方式）但编译器不会报错。

```
public class ThrowExample {
 public static void main(String[] args) {
 try {
 throwException();
 } catch (Exception e) {
 System.out.println("捕获到异常: " + e.getMessage());
 }
 }

 public static void throwException() throws Exception {
 throw new Exception("这是一个自定义异常");
 }
}
```

## 7.40 在finally中return会发生什么？

在Java中，如果在finally块中使用return语句，那么它将覆盖try或catch块中的任何返回值。这是因为finally块的执行是在try和catch块之后进行的，无论try和catch块是否成功完成或者抛出异常。

```
public class ReturnInFinallyExample {
 public static void main(String[] args) {
 System.out.println(testReturnInFinally());
 }
}
```

```

public static int testReturnInFinally() {
 try {
 // 这里可能会发生异常
 int result = 10 / 0; // 这将抛出ArithmeticException
 return result;
 } catch (ArithmeticException e) {
 System.out.println("捕获到异常: " + e);
 return -1;
 } finally {
 // 这里的return将覆盖上面的return结果
 return 20;
 }
}
}
20

```

当Java程序执行try块、catch块时遇到了return或throw语句，这两个语句都会导致该方法立即结束，但是系统执行这两个语句并不会结束该方法，而是去寻找该异常处理流程中是否包含finally块，如果没有finally块，程序立即执行return或throw语句，方法终止；如果有finally块，系统立即开始执行finally块。只有当finally块执行完成后，系统才会再次跳回来执行try块、catch块里的return或throw语句；如果finally块里也使用了return或throw等导致方法终止的语句，finally块已经终止了方法，系统将不会跳回去执行try块、catch块里的任何代码。

## 7.泛型

### 0.什么是泛型

- 类型参数**：在定义泛型类或方法时，你可以指定一个或多个类型参数，通常使用大写字母（如 `T`，`E`，`K` 等）表示。这些字母代表待定的具体类型，可以在实例化泛型类或调用泛型方法时确定。
- 实例化**：当你使用泛型类或方法时，需要为每个类型参数提供具体的类型。例如，如果你有一个泛型类 `List<T>`，你可以实例化一个 `List<String>` 或 `List<Integer>`。
- 类型安全性**：泛型的一个主要优势是在编译时确保类型安全。这意味着你不能将不正确的类型放入泛型容器中，这有助于防止运行时错误。

### 1.泛型擦除是什么？

当把一个具有泛型信息的对象赋给另一个没有泛型信息的变量时，所有在尖括号之间的类型信息都将被扔掉。比如一个 `List` 类型被转换为 `List`，则该 `List` 对集合元素的类型检查变成了泛型参数的上限（即 `Object`）。

上述规则即为泛型擦除，可以通过下面代码进一步理解泛型擦除：

```

List<String> list1 = ...;
List list2 = list1; // list2将元素当做Object处理

```

因为泛型其实只是在编译器中实现的而虚拟机并不认识泛型类项，所以要在虚拟机中将泛型类型进行擦除。也就是说，**在编译阶段使用泛型，运行阶段取消泛型，即擦除**。擦除是将泛型类型以其父类代替，如 `String` 变成了 `Object` 等。其实在使用的时候还是进行带强制类型的转化，只不过这是比较安全的转换，因为在编译阶段已经确保了数据的一致性。

```
ArrayList<User> users = new ArrayList<>();
ArrayList<Dog> dogs = new ArrayList<>();
System.out.println(users.getClass() == dogs.getClass()); //true
```

如上，运行结果表面两个list是相等的，因为经过编译后，泛型被擦除，两个list当然也就相等；

## 2. List<? super T>和List<? extends T>有什么区别？

- List<? super T> 用于设定类型通配符的下限，此处 ? 代表一个未知的类型，但它必须是T的父类型；
  - 继续以Number为例，允许的类型可以是Number本身，或者是Object（所有类的根类）
- List<? extends T> 用于设定类型通配符的上限，此处 ? 代表一个未知的类型，但它必须是T的子类型。

- 如果T代表Number类，那么允许的具体类型可以是Number、Integer、Float等。

- ```
List<? extends Number> list = new ArrayList<Number>();
List<? extends Number> list = new ArrayList<Integer>(); //Integer是
Number的子类
List<? extends Number> list = new ArrayList<Float>(); //Float也是Number
的子类
```

8.编译时类型和运行时类型

1.什么是编译时类型和运行时类型？

在编译阶段中，Java源代码通过前端编译器转换成字节码文件，即.class文件。而在运行阶段，这些字节码文件会被Java虚拟机（JVM）加载并执行。

编译时类型（Compile-time Type）

编译时类型是指在编译阶段确定的对象类型。这是由代码中的声明决定的，例如变量声明、方法参数或返回类型的声明。编译器使用这些类型来执行类型检查，确保代码符合Java语言的语法规则和类型系统规则。如果代码违反了这些规则，编译器将生成错误，并且程序无法编译成功。

运行时类型（Run-time Type）

运行时类型是指对象在程序实际执行时的实际类型。在运行时，对象的真实类型可能与编译时类型不同，尤其是在使用继承和多态的情况下。例如，如果你有一个父类类型的引用指向一个子类实例，那么该引用的编译时类型是父类，但其运行时类型是子类。

```
Animal animal = new Dog(); // 或者 new Cat()
```

在这个例子中：

- `animal` 的**编译时类型**是 `Animal`，因为这就是它在代码中被声明的类型。
- `animal` 的**运行时类型**可能是 `Dog` 或者 `Cat`，具体取决于创建时传入的是哪个子类的实例。

8.反射

1.什么是反射

动态获取的信息以及动态调用对象的方法的功能称为Java语言的反射机制。通过反射你可以获取任意一个类的所有属性和方法，你还可以调用这些方法和属性。

通过反射机制，我们可以实现如下的操作：

- 程序运行时，可以通过反射获得任意一个类的Class对象，并通过这个对象查看这个类的信息；
- 程序运行时，可以通过反射创建任意一个类的实例，并访问该实例的成员；
- 程序运行时，可以通过反射机制生成一个类的动态代理类或动态代理对象。

2.反射应用场景

1. **动态创建对象和调用方法**：在运行时获取类信息并创建对象或调用其方法。
2. **框架和库的开发**：如Spring框架使用反射来实现依赖注入（DI）和面向切面编程（AOP）。
3. **序列化/反序列化**：JSON处理库如Jackson或Gson在序列化和反序列化时会使用反射。
4. Java 中的一大利器 **注解** 的实现也用到了反射。为什么你使用 Spring 的时候，一个 `@Component` 注解就声明了一个类为 Spring Bean 呢？为什么你通过一个 `@value` 注解就读取到配置文件中的值呢？究竟是怎么起作用的呢？

这些都是因为你可以基于反射分析类，然后获取到类/属性/方法/方法的参数上的注解。你获取到注解之后，就可以做进一步的处理。
5. 正是因为反射，你才能这么轻松地使用各种框架。像 Spring/Spring Boot、MyBatis 等等框架中都大量使用了反射机制。**这些框架中也大量使用了动态代理，而动态代理的实现也依赖反射。**

3.反射的优缺点？

反射可以让我们的代码更加灵活、为各种框架提供开箱即用的功能提供了便利。

不过，反射让我们在运行时有了分析操作类的能力的同时，也增加了安全问题，比如可以无视泛型参数的安全检查（泛型参数的安全检查发生在编译时）。另外，反射的性能也要稍差点，不过，对于框架来说实际是影响不大的。

反射的优点：

- **可扩展性**：应用程序可以利用全限定名创建可扩展对象的实例，来使用来自外部的用户自定义类。
- **类浏览器和可视化开发环境**：一个类浏览器需要可以枚举类的成员。可视化开发环境（如 IDE）可以从利用反射中可用的类型信息中受益，以帮助程序员编写正确的代码。
- **调试器和测试工具**：调试器需要能够检查一个类里的私有成员。测试工具可以利用反射来自动地调用类里定义的可被发现的 API 定义，以确保一组测试中有较高的代码覆盖率。

反射的缺点：

尽管反射非常强大，但也不能滥用。如果一个功能可以不用反射完成，那么最好就不用。在我们使用反射技术时，下面几条内容应该牢记于心。

- **性能开销**：反射涉及了动态类型的解析，所以 JVM 无法对这些代码进行优化。因此，反射操作的效率要比那些非反射操作低得多。我们应该避免在经常被执行的代码或对性能要求很高的程序中使用反射。
- **安全限制**：使用反射技术要求程序必须在一个没有安全限制的环境中运行。如果一个程序必须在有安全限制的环境中运行，如 Applet，那么这就是个问题。
- **内部暴露**：由于反射允许代码执行一些在正常情况下不被允许的操作（比如访问私有的属性和方法），所以使用反射可能会导致意料之外的副作用，这可能导致代码功能失调并破坏可移植性。反射代码破坏了抽象性，因此当平台发生改变的时候，代码的行为就有可能也随着变化。

9.代理

在Java中，代理（Proxy）设计模式是一种创建型模式，它提供了一个对象来控制对另一个对象的访问。代理模式可以实现额外的功能，比如日志记录、性能监控、权限控制等，而无需修改原有对象的代码（功能增强）。根据实现方式的不同，Java中的代理可以分为静态代理和动态代理。

1.静态代理

静态代理是在编译时期就定义了代理类，代理类和被代理类通常共享一个接口或者抽象类，这样客户端可以通过统一的接口来操作代理类或真实对象。静态代理需要手动编写代理类，因此比较繁琐，但其优点是代理类可以在编译时检查类型错误

静态代理实现步骤：

1. 定义一个接口及其实现类；
2. 创建一个代理类同样实现这个接口
3. 将目标对象注入进代理类，然后在代理类的对应方法调用目标类中的对应方法。这样的话，我们就可以通过代理类屏蔽对目标对象的访问，并且可以在目标方法执行前后做一些自己想做的事情

静态代理优缺点：

优点：

通过静态代理，我们达到了功能增强的目的，而且没有侵入原代码，这是静态代理的一个优点。静态代理实现简单，且不侵入原代码。

缺点

- 1、 当需要代理多个类的时候，由于代理对象要实现与目标对象一致的接口，有两种方式：
 - 只维护一个代理类，由这个代理类实现多个接口，但是这样就导致代理类过于庞大
 - 新建多个代理类，每个目标对象对应一个代理类，但是这样会产生过多的代理类
- 2、 当接口需要增加、删除、修改方法的时候，目标对象与代理类都要同时修改，不易维护。

示例代码

```
// 定义一个接口
interface Subject {
    void doSomething();
}

// 目标类实现接口
class RealSubject implements Subject {
    @Override
    public void doSomething() {
        System.out.println("RealSubject is doing something");
    }
}

// 代理类实现接口
class StaticProxy implements Subject {
    private RealSubject realSubject;

    public StaticProxy(RealSubject realSubject) {
        this.realSubject = realSubject;
    }
}
```

```

    }

    @Override
    public void doSomething() {
        System.out.println("Before calling RealSubject's method");
        realSubject.doSomething();
        System.out.println("After calling RealSubject's method");
    }
}

public class Main {
    public static void main(String[] args) {
        RealSubject realSubject = new RealSubject();
        StaticProxy staticProxy = new StaticProxy(realSubject);
        staticProxy.doSomething();
    }
}

```

2.动态代理

1.JDK动态代理

如果目标类实现了接口，Spring AOP会选择使用JDK动态代理目标类。代理类根据目标类实现的接口动态生成，不需要自己编写，生成的动态代理类和目标类都实现相同的接口。JDK动态代理的核心是 `InvocationHandler` 接口和 `Proxy` 类。

缺点：**目标类必须有实现的接口**。如果某个类没有实现接口，那么这个类就不能用JDK动态代理。

2.CGLIB动态代理

通过继承实现。如果目标类没有实现接口，那么Spring AOP会选择使用CGLIB来动态代理目标类。CGLIB（Code Generation Library）可以在运行时动态生成类的字节码，动态创建目标类的子类对象，在子类对象中增强目标类。

优点：目标类不需要实现特定的接口，更加灵活。

缺点：CGLIB是通过继承的方式做的动态代理，因此如果某个类被标记为 `final`，那么它是无法使用CGLIB做动态代理的。

3.静态代理和动态代理的对比)

1. **灵活性**：动态代理更加灵活，不需要必须实现接口，可以直接代理实现类，并且可以不需要针对每个目标类都创建一个代理类。另外，静态代理中，接口一旦新增加方法，目标对象和代理对象都要进行修改，这是非常麻烦的！
2. **JVM 层面**：静态代理在编译时就将接口、实现类、代理类这些都变成了一个个实际的 class 文件。而动态代理是在运行时动态生成类字节码，并加载到 JVM 中的。

4.JDK动态代理代码举例

```

import java.lang.reflect.InvocationHandler;
import java.lang.reflect.Method;
import java.lang.reflect.Proxy;
import java.util.Date;

//抽象主题类
interface Subject {

```

```

        void doSomething();
    }
    //真实主题角色
    class RealSubject implements Subject{
        @Override
        public void doSomething() {
            //TODO 具体执行的事
        }
    }

    class DynamicProxyHandler implements InvocationHandler {
        Object target; // 被代理的对象，实际的方法执行者

        public DynamicProxyHandler(Object target) {
            this.target = target;
        }
        @Override
        public Object invoke(Object proxy, Method method, Object[] args) throws
        Throwable {
            before();
            Object result = method.invoke(target, args); // 调用 target 的 method 方
            法
            after();
            return result; // 返回方法的执行结果
        }
        // 调用invoke方法之前执行
        private void before() {
        }
        // 调用invoke方法之后执行
        private void after() {
        }
    }

    //测试
    public class Test {
        public static void main(String[] args) {
            // 1、创建具体主题类
            RealSubject realSubject = new RealSubject();

            // 2、获取对应的invocationHandler（用于处理方法调用的处理器）
            //创建一个动态代理处理器`DynamicProxyHandler`的实例，并将`userServiceImpl`作为
            参数传递给它。这个处理器将负责处理方法调用。
            InvocationHandler invocationHandler = new
            DynamicProxyHandler(realSubject);

            // 3. 获取对应的 ClassLoader（用于加载代理类的类加载器）
            ClassLoader classLoader = realSubject.getClass().getClassLoader();

            // 4. 获取对应的interfaces（代理对象需要实现的接口数组），这里的UserServiceImpl只
            实现了一个接口UserService
            Class[] interfaces = realSubject.getClass().getInterfaces();

            //5.创建Handler
            DynamicProxyHandler proxyHandler = new DynamicProxyHandler(realSubject);
            // 6、动态产生代理类
            //使用`Proxy.newProxyInstance()`方法动态生成代理对象。传入类加载器、接口数组和处理
            器作为参数

```

```
Subject proxy = (Subject) Proxy.newProxyInstance(classLoader, interfaces,
proxyHandler);

//通过代理对象调用`select()`方法。由于使用了动态代理，这个方法实际上是由
`DynamicProxyHandler`中的`invoke()`方法来处理的。
proxy.doSomething();
}
}
```

4.Spring中的代理模式

Spring中AOP的实现.

Spring 的 AOP 是通过**动态代理**实现的。如果我们为 Spring 的某个 bean 配置了切面，那么 Spring 在创建这个 bean 的时候，实际上创建的是这个 bean 的一个代理对象，我们后续对 bean 中方法的调用，实际上调用的是代理类重写的代理方法。而 Spring 的 AOP 使用了两种动态代理，分别是**JDK的动态代理**，以及**CGLib的动态代理**。

9.序列化与反序列化

1.什么是序列化?什么是反序列化?

Java的序列化是将Java对象转换为二进制字节流的过程，而反序列化则是将这个二进制字节流恢复为Java对象的过程。

如果我们需要持久化 Java 对象比如将 Java 对象保存在文件中，或者在网络传输 Java 对象，这些场景都需要用到序列化。

简单来说：

- **序列化**：将数据结构或对象转换成二进制字节流的过程
- **反序列化**：将在序列化过程中所生成的二进制字节流转换成数据结构或者对象的过程

示例代码

```
import java.io.*;

class Person implements Serializable {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public String toString() {
        return "Person{" +
            "name='" + name + '\'' +
            ", age=" + age +
            '}';
    }
}

public class SerializeDemo {
```

```

public static void main(String[] args) {
    // 创建一个Person对象
    Person person = new Person("张三", 30);

    try {
        // 序列化对象到文件
        FileOutputStream fileOut = new FileOutputStream("person.ser");
        ObjectOutputStream out = new ObjectOutputStream(fileOut);
        out.writeObject(person);
        out.close();
        fileOut.close();
        System.out.println("Serialized data is saved in person.ser");
    } catch (IOException i) {
        i.printStackTrace();
    }

    person = null;

    try {
        // 从文件中反序列化对象
        FileInputStream fileIn = new FileInputStream("person.ser");
        ObjectInputStream in = new ObjectInputStream(fileIn);
        person = (Person) in.readObject();
        in.close();
        fileIn.close();
    } catch (IOException i) {
        i.printStackTrace();
        return;
    } catch (ClassNotFoundException c) {
        System.out.println("Person class not found");
        c.printStackTrace();
        return;
    }

    System.out.println("Deserialized Person:");
    System.out.println(person);
}
}

```

2.怎么实现序列化?

1. 让类实现 `java.io.Serializable` 接口。这个接口是一个标记接口，不包含任何方法，它的存在告诉Java虚拟机 (JVM) 这个类的对象可以被序列化。
2. 使用 `ObjectOutputStream` 类的 `writeObject()` 方法将对象写入输出流。
3. 使用 `ObjectInputStream` 类的 `readObject()` 方法从输入流中读取对象。

2.常见应用场景

下面是序列化和反序列化常见应用场景：

- 对象在进行网络传输（比如远程方法调用 RPC 的时候）之前需要先被序列化，接收到序列化的对象之后需要再进行反序列化；
- 将对象存储到文件之前需要进行序列化，将对象从文件中读取出来需要进行反序列化；

- 将对象存储到数据库（如 Redis）之前需要用到序列化，将对象从缓存数据库中读取出来需要反序列化；
- 将对象存储到内存之前需要进行序列化，从内存中读取出来之后需要进行反序列化。

3.如果有些字段不想进行序列化怎么办? (transient关键字)

Java语言的关键字，变量修饰符，如果用transient声明一个实例变量，当对象存储时，它的值不需要维持。

也就是说被transient修饰的成员变量，在序列化的时候其值会被忽略，在被反序列化后，transient 变量的值被设为初始值，如 int 型的是 0，对象型的是 null。

5.static 属性为什么不会被序列化?

因为序列化是针对对象而言的，而 static 属性优先于对象存在，随着类的加载而加载，所以不会被序列化。

看到这个结论，是不是有人会问，serialVersionUID 也被 static 修饰，为什么 serialVersionUID 会被序列化? 其实 serialVersionUID 属性并没有被序列化，JVM 在序列化对象时会自动生成一个 serialVersionUID，然后将我们显示指定的 serialVersionUID 属性值赋给自动生成的 serialVersionUID

10.I/O流

1.Java IO 流了解吗?

IO 即 Input/output，输入和输出。数据输入到计算机内存的过程即输入，反之输出到外部存储（比如数据库，文件，远程主机）的过程即输出。数据传输过程类似于水流，因此称为 IO 流。IO 流在 Java 中分为输入流和输出流，而根据数据的处理方式又分为字节流和字符流。

Java IO 流的 40 多个类都是从如下 4 个抽象类基类中派生出来的。

- `InputStream/Reader`: 所有的输入流的基类，前者是字节输入流，后者是字符输入流。
- `OutputStream/Writer`: 所有输出流的基类，前者是字节输出流，后者是字符输出流。

2.I/O 流为什么要分为字节流和字符流呢?

问题本质想问：不管是文件读写还是网络发送接收，信息的最小存储单元都是字节，那为什么 I/O 流操作要分为字节流操作和字符流操作呢？

首先明确字节流适用于任何场景，而且有字节缓冲流，能提高读取和输入的效率，也就是 `BufferedOutputStream/BufferedInputStream`。其操作与字节流基本都一样。

而字符流是为了应对汉字出现的情况。在GBK中汉字占2个字节，在UTF-8中汉字占3个字节，所以我们通过字节流读取文件的时候一般都是逐个字节转换就会导致乱码，而手动去根据不同编码去拼接则不方便，所以有字符流。

3.I/O模型

见计网操作系统笔记

11.常见的关键字

final

1. **基本数据类型**用final修饰，则不能修改，是常量；**对象引用**用final修饰，则引用只能指向该对象，不能指向别的对象，但是对象本身可以修改。
2. final修饰的方法不能被子类重写
3. final修饰的类不能被继承。

this

`this.属性名称` 指访问类中的成员变量，可以用来区分成员变量和局部变量。

super

super 关键字用于在子类中访问父类的变量和方法。

static和final有什么区别？

具体分析如下：

1. **代码块**
 - **static**：可以修饰静态代码块，该代码块在类加载的时候执行，常用于初始化静态变量。
 - **final**：不可以修饰代码块，因为final关注的是不变性而不是初始化过程。
2. **局部变量**
 - **static**：不能修饰局部变量。静态变量是类级别的，而局部变量属于方法的范畴，与类无关。
 - **final**：可以修饰局部变量，表示该局部变量在声明后不能再被重新赋值。
3. **重写**
 - **static**：静态方法不能被重写。
 - **final**：被final修饰的方法不能被子类重写，确保了方法的不变性。
4. **访问方式**
 - **static**：静态成员可以在不创建类的实例的情况下通过类名直接访问。
 - **final**：被final修饰的成员也可以通过类名访问，但是其关键属性在于值的不可变性。

12.Java8的新特性有哪些？

Java 8的新特性主要包括**Lambda表达式**、**函数式接口**、**方法引用**、**接口的静态方法和默认方法**等。具体介绍如下：

1. **Lambda表达式和函数式接口编程**：Lambda 表达式是一种可以创建匿名函数的方法。它允许你将函数作为参数传递给方法，或者将函数存储在变量中

- ```
Arrays.asList("a", "b", "d").forEach(e -> System.out.println(e));
```

```

○ 1import java.util.Arrays;
 2import java.util.List;
 3
 4public class LambdaExample {
 5 public static void main(String[] args) {
 6 List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
 7
 8 // 使用 Lambda 表达式来过滤出偶数
 9 numbers.stream()
 10 .filter(n -> n % 2 == 0)
 11 .forEach(System.out::println);
 12 }
 13}

```

```

○ import java.util.Arrays;
 import java.util.List;
 import java.util.stream.Collectors;

 public class StreamExample {
 public static void main(String[] args) {
 // 创建一个字符串列表
 List<String> names = Arrays.asList("张三", "李四", "王五", "赵六",
 "孙七");

 // 使用Stream流过滤出长度大于2的名字，并将它们转换为大写
 List<String> filteredNames = names.stream()
 .filter(name -> name.length() > 2)
 .map(String::toUpperCase)
 .collect(Collectors.toList());

 // 输出过滤后的名字列表
 System.out.println(filteredNames);
 }
 }

```

2. **方法引用**：方法引用则是另一种简化对方法的调用的方式，它允许直接引用已有的方法或构造器，而不是在其内部再创建Lambda表达式。这大大减少了代码量，并提高了代码的可读性。

3. **Stream API**：Stream API是Java 8中加入的一个全新的数据处理工具，它允许以声明性方式处理数据集，就像对数据库查询一样。Stream 使用一种类似用 SQL 语句从数据库查询数据的直观方式来提供一种对 Java 集合运算和表达的高阶抽象。

```
○ 1import java.util.Arrays;
 2import java.util.List;
 3
 4public class LambdaExample {
 5 public static void main(String[] args) {
 6 List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
 7
 8 // 使用 Lambda 表达式来过滤出偶数
 9 numbers.stream()
 10 .filter(n -> n % 2 == 0)
 11 .forEach(System.out::println);
 12 }
 13}
```

## 13.Java19新特性虚拟线程

### 1.什么是虚拟线程？

虚拟线程 (Virtual Thread) 是 JDK 而不是 OS 实现的轻量级线程(Lightweight Process, LWP) , 由 JVM 调度。许多虚拟线程共享同一个操作系统线程, 虚拟线程的数量可以远大于操作系统线程的数量。

### 2.虚拟线程有什么优点和缺点？

#### 优点

- 非常轻量级：可以在单个线程中创建成百上千个虚拟线程而不会导致过多的线程创建和上下文切换。
- 简化异步编程：虚拟线程可以简化异步编程，使代码更易于理解和维护。它可以将异步代码编写得更像同步代码，避免了回调地狱 (Callback Hell) 。
- 减少资源开销：相比于操作系统线程，虚拟线程的资源开销更小。本质上是提高了线程的执行效率，从而减少线程资源的创建和上下文切换。

#### 缺点

- 不适用于计算密集型任务：虚拟线程适用于 I/O 密集型任务，但不适用于计算密集型任务，因为密集型计算始终需要 CPU 资源作为支持。
- 依赖于语言或库的支持：协程需要编程语言或库提供支持。不是所有编程语言都原生支持协程。比如 Java 实现的虚拟线程。

### 3.虚拟线程的创建方法

1. 使用 `Thread.startVirtualThread()` 创建
2. 使用 `Thread.ofVirtual()` 创建
3. 使用 `ThreadFactory` 创建

## 14.并发类

### 1.FutureTask

**FutureTask是一个实现了 `Future` 接口和 `Runnable` 接口的类，用于异步执行计算任务并获取结果。**

**FutureTask通过实现`Callable`和`Runnable`接口，允许将计算任务提交给`ExecutorService`进行异步执行。**这种机制使得FutureTask非常适合用在需要并行处理或非阻塞性IO操作的场景。

FutureTask的内部状态由一个私有的状态变量维护，这个状态可以标识任务处于等待、运行、完成或取消等不同阶段。通过这个状态变量，可以判断任务是否已经完成，并且能够获取任务的结果或者异常信息。

使用FutureTask时，可以通过构造函数传入一个Callable或Runnable对象。**如果传入Callable对象，FutureTask在执行完后可以获取返回值；如果传入Runnable对象，则不返回任何结果。提交给ExecutorService执行后，通过调用FutureTask的get()方法可以获取计算结果。这个方法会阻塞调用线程直到任务完成，并且能够返回计算的结果，或者在任务执行异常时抛出ExecutionException异常。**

另外，FutureTask还提供了cancel()方法来尝试取消任务的执行。该方法接受一个布尔型参数，当参数为true时，如果任务正在运行，则会试图中断它；如果任务还未开始，将其标记为取消状态。

值得注意的是，FutureTask无法直接获取任务的执行进度，只能通过isDone()方法检查任务是否完成。同时，每个FutureTask都是一次性使用的，一旦完成任务便不能再被重置或再次使用。

综上所述，FutureTask不仅在理论上提供了强大的异步执行能力，而且在实际应用中具有广泛的适用性。例如，它可以用于实现简单的缓存机制，当缓存中不存在指定值时，通过FutureTask来计算并缓存结果；也可以用于并发任务的控制和管理，如等待所有任务完成或只等待其中一个任务完成。合理利用FutureTask能够显著提高程序的并发性能和效率。

```
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.FutureTask;

public class MyCallable implements Callable<Integer> {
 //MyCallable类实现了Callable<String>接口，该接口是一个泛型接口，指定了call()方法的返回类型为String
 @Override
 public Integer call() { //call方法可以抛出异常
 int i=0;
 for (i = 0; i <5 ; i++) {
 System.out.println(Thread.currentThread().getName()+"====="+i);
 }
 return i;
 }

 public static void main(String[] args) throws ExecutionException,
 InterruptedException {

 // 创建MyCallable对象
 MyCallable mc = new MyCallable() ;

 //创建FutureTask<String>对象ft，并将mc作为参数传入构造方法，用于封装可调用对象
 FutureTask<Integer> task = new FutureTask<Integer>(mc);
 //通过将Callable对象封装在FutureTask中，可以在多线程环境下执行任务，并获取任务执行结果。

 for (int i = 0; i <3 ; i++) {
 System.out.println(Thread.currentThread().getName()+"====="+i);
 if(i==0){
 Thread t1=new Thread(task,"我是fu线程");
 t1.start();
 }
 }
 }
}
```

```
 }
 }
 //得到返回值，注意这个必须要有显示抛出异常
 try{
 System.out.println("子线程的返回值"+task.get());
 System.out.println("子线程的返回值"+task.get());
 }catch (Exception e){
 e.printStackTrace();
 }

 }
}
main=====0
main=====1
main=====2
我是fu线程=====0
我是fu线程=====1
我是fu线程=====2
我是fu线程=====3
我是fu线程=====4
子线程的返回值5
子线程的返回值5
```